

Práctica Guiada: Gestor de Stock para una Tienda de Cómic

Construir una aplicación en Angular que gestione el inventario de una tienda. Aplicaremos variables, **Signals**, **Interpolación**, **Event Binding** y **Control Flow (@if / @for)**.

Preparación del terreno

1. Abre el terminal en Visual Studio Code.
2. Crea un nuevo componente ejecutando:

```
Bash
ng new tienda-comics //crea el proyecto
cd tienda-comics //entramos en el proyecto
ng g c comic-manager //crea el componente
```

3. Abre el archivo comic-manager.component.ts y el archivo comic-manager.component.html. Ponlos en pantalla dividida.

Definir el "molde" y los datos iniciales (.ts)

Antes de mostrar nada en pantalla, necesitamos definir la estructura de un Cómic y crear los datos iniciales.

1. **Crea la interfaz:** En comic-manager.component.ts, justo encima de @Component, define qué campos tendrá cada cómic:

```
TypeScript
interface Comic {
  id: number;
  titulo: string;
  precio: number;
  stock: number;
}
```

2. **Importa la función signal:** Asegúrate de que arriba del todo tienes el import:

```
TypeScript
import { Component, signal } from '@angular/core';
```

3. **Crea las Signals dentro de la clase:**

- Una Signal para saber si estamos en modo compacto (boolean).
- Una Signal para guardar la lista de cómics (Comic[]).

TypeScript

```
export class ComicManagerComponent {  
  // Signal para controlar el modo de vista  
  mostrarModoCompacto = signal<boolean>(false);  
  
  // Signal con el listado inicial de cómics  
  comics = signal<Comic[]>([  
    { id: 101, titulo: 'The Amazing Spider-Man #1', precio: 15.99, stock: 3 },  
    { id: 102, titulo: 'Batman: The Dark Knight Returns', precio: 22.50, stock: 0 },  
    { id: 103, titulo: 'Watchmen', precio: 18.00, stock: 5 }  
  ]);  
}
```

Pintar la lista con `@for` y `track` (.html)

Ahora vamos a mostrar el catálogo en la pantalla.

1. Ve a `comic-manager.component.html` y borra el contenido por defecto.
2. Usa la estructura **`@for`** para recorrer la Signal `comics()`.
3. Recuerda incluir el campo único en **`track`** (`comic.id`) e interpolar con `{{ }}`.

HTML

```
<h2>Tienda de Cómics</h2>  
  
<div class="contenedor-comics">  
  @for (comic of comics(); track comic.id) {  
    <div style="border: 1px solid #ccc; margin: 10px; padding: 10px;">  
      <h3>{{ comic.titulo }}</h3>  
      <p>Precio: {{ comic.precio }} €</p>  
      <p>Stock disponible: {{ comic.stock }} unidades</p>  
    </div>  
  } @empty {  
    <p>No hay cómics registrados en el inventario.</p>  
  }  
</div>
```

Guarda ambos archivos y comprueba en el navegador que ves los 3 cómics listados.

Añadir lógica condicional con `@if` (.html)

Queremos que si el stock de un cómic llega a 0, aparezca un mensaje destacado de **AGOTADO**.

1. Dentro de cada tarjeta de cómic en el `.html`, añade un bloque `@if / @else`:

HTML

```
<!-- Justo debajo del texto de stock -->
@if (comic.stock > 0) {
  <span style="color: green;"> En stock</span>
} @else {
  <span style="color: red; font-weight: bold;"> ¡AGOTADO!</span>
}
```

Dar interactividad con Event Binding (.ts + .html)

Vamos a hacer que los datos reaccionen cuando el usuario haga clic en los botones.

1. **En el .ts**, añade los métodos para modificar el stock usando .update():

TypeScript

```
aumentarStock(idComic: number): void {
  this.comics.update(listaActual =>
    listaActual.map(item =>
      item.id === idComic ? { ...item, stock: item.stock + 1 } : item
    )
  );
}

venderComic(idComic: number): void {
  this.comics.update(listaActual =>
    listaActual.map(item =>
      item.id === idComic && item.stock > 0 ? { ...item, stock: item.stock - 1 } : item
    )
  );
}
```

2. **En el .html**, conecta los métodos a botones usando (**click**):

HTML

```
<br><br>
<button (click)="aumentarStock(comic.id)"> Añadir 1</button>
<button (click)="venderComic(comic.id)" [disabled]="comic.stock === 0">  Vender 1</button>
```

Haz clic en "Vender 1" en el primer cómic hasta que llegue a 0. Verifica si la etiqueta cambia automáticamente a "¡AGOTADO!" sin recargar la página.

Alternar vistas de la interfaz (.ts + .html)

Por último, añadiremos un botón principal que cambie entre una vista detallada y una vista simplificada.

1. En el .ts, añade la función para alternar la Signal booleana:

```
TypeScript
toggleVista(): void {
  this.mostrarModoCompacto.update(estadoActual => !estadoActual);
}
```

2. En el .html, envuelve la vista con un @if general basado en mostrarModoCompacto() y añade el botón superior:

```
HTML
<h2>Tienda de Cómic</h2>

<button (click)="toggleVista()">
  Cambiar a vista {{ mostrarModoCompacto() ? 'Detallada' : 'Compacta' }}
</button>

<hr>

@if (mostrarModoCompacto()) {
  <!-- Vista Compacta (Solo lista) -->
  <ul>
    @for (comic of comics(); track comic.id) {
      <li><strong>{{ comic.titulo }}</strong> - Stock: {{ comic.stock }}</li>
    }
  </ul>
} @else {
  <!-- Vista Detallada (Tarjetas completas con botones) -->
  <!-- (Aquí va el contenedor con tarjetas que hicimos en las misiones anteriores) -->
}
```

Vaciar almacén

Añade un botón al final del HTML llamado "**Vaciar Almacén**" que use el método .set() para poner la Signal de cómics como una lista vacía []:

- ¿Qué debería mostrarse en pantalla cuando le das a ese botón?